Milestone 7: RAG Pipeline Implementation – Gradient Descenters

Retrieval Augmented Generation for Database Agent Application

The RAG pipeline in our application is setup to enhance the SQL Query generation from user provided natural language. The relevant context we are including in our RAG setup include SQL Query examples that have been validated, join patterns, table schemas, along with business rules we want our agent to adhere to.

Our intended result is to use this to provide focused, query-specific context that will improve the SQL generation accuracy, while decreasing result latency and token usage.

Document Selection, Vector Database, Embeddings, and Chunking

64 documents were stored into a ChromaDB vector database. Some of this content was created using a data dictionary for the AdventureWorks2025DW database found online. Our documents include

- 22 validated SQL Examples
    - These serve as the gold-standard query answer pairs, and they provide few-shot learning examples for the LLM to use
- 7 Join patterns
    - Table relationship templates which will prevent join errors and provide context. Used to encode complex dimensional modeling relationships between fact and dimension tables.
- 14 Table Schemas
    - Column definitions and data types, enabling accurate column selection and filtering. Selected the relevant Fact (4) and Dimension tables (10)
- 21 Business Rules
    - These provide metric definitions and constraints as learned from the data dictionary. They ensure compliance with business logic and privacy rules.

OpenAI "text-embedding-3-small" was chosen for our embedding model, as it produces dense vector representations that capture query intent.

We chose ChromaDB as our vector database since it has a collection-based organization for our 4 collections (examples, join patterns, schemas, and business rules).

For our chunking strategy, we did this at the document level as described in the 64 documents above. Each SQL example, table schema, and join is its own document. Business rules are stored within one document.

Retrieval Mechanisms

Retrieval was done using K-Nearest-Neighbors with cosine similarity. The retrieval was set to extract the Top-K from each of our collections. K_examples = 3 (for the most similar SQL examples), k_patterns = 2 (For the most relevant join patterns), k_schemas = 5, and so on.

This selective retrieval is more useful than the previous baseline approach we had of providing all 31 database tables to the agent every time. This will help reduce noise to the LLM, help it identify the correct tables faster, decrease token consumption, and minimize hallucinations.

For similarity scoring, each retrieved document includes a distance metric, where the lower they are the more similar and a similarity score which the higher it is, the more relevant.

Context Integration for Response Quality

The RAG enhanced prompt retrieved the most relevant SQL examples, most relevant join patterns, most relevant table schemas, and all of the business rules. This is more dynamic and context driven compared to the baseline prompt we were using before implementing the RAG pipeline.

Evaluation & Metrics

We found that of the 12 test cases we evaluated retrieval quality for (moderate complexity), the result

- Precision@5 = 1.0
- Recall = 0.889
- F1 Score = 0.928
- Perfect Retrievals = 9/12

We found from accuracy standpoint, the RAG system slightly outperformed our baseline model (12/15 successful queries compared to 11/15). The failed queries were ones that were very complex involving window functions.

The real area where the RAG pipeline shined was in latency and token usage. We saw a decrease in token usage by 77%, and average latency reduction of 29%, both which are significant improvements when considering that a database agent would be used by many individuals in a sales organization. Implementing the RAG pipeline would significantly reduce cost and improve performance.

Appendix:

RAG Documentation

SQL Examples. 1 of 22 SQL examples


```
{
  "id": 12,
  "category": "Group By",
  "fact_table": "FactInternetSales",
  "natural_language": "Show me total sales by country",
  "sql": "SELECT\n    g.EnglishCountryRegionName AS Country,\n    SUM(fis.SalesAmount) AS TotalRevenue,\n    COUNT(DISTINCT fis.CustomerKey) AS UniqueCustomers,\n    COUNT(DI
  "expected_tables": ["FactInternetSales", "DimCustomer", "DimGeography"],
  "difficulty": "Hard",
  "description": "GROUP BY with multiple joins through customer geography"
},
```


Business Rules (metric definitions, privacy, customer segmentation, product classifications, time periods, geography, and naming convention rules). Screenshot is just a small portion of this

```
==============================================
METRIC DEFINITIONS
==============================================

Revenue / Sales:
  - Definition: Total amount charged to customers including tax
  - Source: SalesAmount column from FactInternetSales or FactResellerSales
  - Formula: SUM(SalesAmount)
  - Includes: Product price + tax + freight
  - Currency: Values are in USD (or use CurrencyKey for multi-currency)

Net Revenue:
  - Definition: Sales amount minus tax
  - Formula: SUM(SalesAmount - TaxAmt)
  - Use when: Customer wants revenue excluding tax

Profit / Gross Profit:
  - Definition: Sales minus product cost
  - Formula: SUM(SalesAmount - TotalProductCost)
  - Note: Does NOT include operating expenses, only COGS (cost of goods sold)

Profit Margin:
  - Definition: Profit as percentage of sales
  - Formula: (SUM(SalesAmount - TotalProductCost) / NULLIF(SUM(SalesAmount), 0)) * 100
  - Format: Express as percentage (e.g., 45.2% margin)
  - Use NULLIF to avoid division by zero

Units Sold / Quantity:
  - Definition: Total number of items sold
  - Formula: SUM(OrderQuantity)
  - Source: OrderQuantity from fact tables

Average Order Value (AOV):
  - Definition: Revenue per order
  - Formula: SUM(SalesAmount) / NULLIF(COUNT(DISTINCT SalesOrderNumber), 0)
  - Critical: Use DISTINCT on SalesOrderNumber
  - Why: Multi-line orders would inflate average without DISTINCT

Order Count:
  - Definition: Number of unique orders
  - Formula: COUNT(DISTINCT SalesOrderNumber)
  - Critical: Must use DISTINCT since orders can have multiple lines
```

Glossary of terms used in database (just a section of a total glossary taken from data dictionary)

```
==============================================
SALES & REVENUE TERMS
==============================================

Revenue:
  Total SalesAmount from sales transactions. Includes product price, tax, and freight.

Internet Sales:
  Direct-to-consumer online orders stored in FactInternetSales table.

Reseller Sales:
  Business-to-business wholesale orders stored in FactResellerSales table.
  Resellers purchase in bulk and resell to end consumers.

Sales Amount:
  Total charged to customer for an order line including tax and freight.

Order:
  A single purchase transaction identified by unique SalesOrderNumber.
  Can contain multiple line items (products).

Order Line:
  Individual product within an order. One order can have many lines.

Sales Channel:
  Method of sale - either Internet (B2C) or Reseller (B2B).
```

Join patterns (1 section of 1 join pattern, 7 total)

```
PATTERN: Top N Ranking Queries

WHEN TO USE:
- "Top 10", "Top 5", "Best", "Highest", "Most", "Leading"
- "Bottom", "Worst", "Lowest", "Least" (use ASC instead of DESC)
- Ranking or leaderboard queries

SYNTAX:
SELECT TOP n
    [columns]
FROM [tables]
[WHERE conditions]
GROUP BY [grouping columns]
ORDER BY [ranking metric] DESC

CRITICAL RULES:
1. TOP must come immediately after SELECT
2. ORDER BY is REQUIRED with TOP
3. For aggregations, include key in GROUP BY even if not in SELECT
4. Use DESC for "top/best/highest", ASC for "bottom/worst/lowest"

EXAMPLE - Top 10 Products by Revenue:
SELECT TOP 10
    p.EnglishProductName,
    SUM(fis.SalesAmount) AS TotalRevenue,
    SUM(fis.OrderQuantity) AS TotalUnitsSold,
    COUNT(DISTINCT fis.SalesOrderNumber) AS OrderCount
FROM dbo.FactInternetSales fis
INNER JOIN dbo.DimProduct p ON fis.ProductKey = p.ProductKey
GROUP BY p.EnglishProductName, p.ProductKey
ORDER BY TotalRevenue DESC
```

Schema (1 section of 1 schema, 14 total). Taken from Data Dictionary

```
TABLE: dbo.FactInternetSales
======================================================================

DESCRIPTION:
Online/internet sales transactions (B2C). Each row represents one line item from a customer order. Primary fact table for direct-to-consumer sales analysis.

TYPE: Fact Table

======================================================================
COLUMNS:
======================================================================

ProductKey
  Type: int NOT NULL [FK → dbo.DimProduct.ProductKey]

OrderDateKey
  Type: int NOT NULL [FK → dbo.DimDate.DateKey]

DueDateKey
  Type: int NOT NULL [FK → dbo.DimDate.DateKey]

ShipDateKey
  Type: int NOT NULL [FK → dbo.DimDate.DateKey]

CustomerKey
  Type: int NOT NULL [FK → dbo.DimCustomer.CustomerKey]

PromotionKey
  Type: int NOT NULL [FK → dbo.DimPromotion.PromotionKey]

CurrencyKey
  Type: int NOT NULL [FK → dbo.DimCurrency.CurrencyKey]

SalesTerritoryKey
  Type: int NOT NULL [FK → dbo.DimSalesTerritory.SalesTerritoryKey]

SalesOrderNumber
  Type: nvarchar(20.0) NOT NULL [PRIMARY KEY]

SalesOrderLineNumber
  Type: tinyint NOT NULL [PRIMARY KEY]

RevisionNumber
  Type: tinyint NOT NULL

OrderQuantity
```

```
======================================================================
FOREIGN KEY RELATIONSHIPS:
======================================================================

  CurrencyKey → dbo.DimCurrency.CurrencyKey
  CustomerKey → dbo.DimCustomer.CustomerKey
  DueDateKey → dbo.DimDate.DateKey
  OrderDateKey → dbo.DimDate.DateKey
  ProductKey → dbo.DimProduct.ProductKey
  PromotionKey → dbo.DimPromotion.PromotionKey
  SalesTerritoryKey → dbo.DimSalesTerritory.SalesTerritoryKey
  ShipDateKey → dbo.DimDate.DateKey

======================================================================
USAGE NOTES:
======================================================================

- Primary fact table for internet/online sales analysis
- Join to DimProduct, DimCustomer, DimDate for complete analysis
- OrderDateKey is primary date for sales (not ShipDateKey)
- SalesAmount includes tax and freight
- Use DISTINCT on SalesOrderNumber for order counting
```

Vector Database Loading

```
🔄 Initializing RAG system...

✓ RAG Manager initialized with persist directory: ./chroma_db
Loading documents into RAG system...
======================================================================
✓ Loaded 22 SQL examples
✓ Loaded 7 join patterns
✓ Loaded 14 table schemas
✓ Loaded 21 business rule documents
======================================================================
✓ RAG system fully initialized!
✓ Total documents loaded: 64
  - SQL Examples: 22
  - Join Patterns: 7
  - Table Schemas: 14
  - Business Rules: 21

======================================================================
RAG SYSTEM STATISTICS
======================================================================
Total Documents Loaded: 64

Breakdown by Collection:
  • SQL Examples:      22 documents
  • Join Patterns:      7 documents
  • Table Schemas:     14 documents
  • Business Rules:    21 documents
======================================================================
```

Retrieval

```
======================================================================
USER QUERY: 'Show me total sales by product category'
======================================================================

🔄 RETRIEVED CONTEXT:

1️⃣  SIMILAR SQL EXAMPLES (k=3):
----------------------------------------------------------------------
2️⃣  RELEVANT JOIN PATTERNS (k=2):
----------------------------------------------------------------------
3️⃣  RELEVANT TABLE SCHEMAS (k=5):
----------------------------------------------------------------------
   Table 1: DimProductCategory (dimension)
   Similarity: 0.414 | Distance: 0.586

   Table 2: DimProductSubcategory (dimension)
   Similarity: 0.391 | Distance: 0.609

   Table 3: FactInternetSales (fact)
   Similarity: 0.348 | Distance: 0.652

   Table 4: FactResellerSales (fact)
   Similarity: 0.337 | Distance: 0.663

   Table 5: DimProduct (dimension)
...
----------------------------------------------------------------------
======================================================================
```

SQL generation with RAG

```
Query 1: Show me total sales by product category
──────────────────────────────────────────────────────────────────────────────
c:\Users\Vamsi Chintalapati\Desktop\DSBA 6010\final_project\database_utils.py:54: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlit
  df = pd.read_sql(sql, self.connection)

✨ RAG Generated SQL:
SELECT
    pc.EnglishProductCategoryName,
    SUM(isales.SalesAmount) AS TotalSales
FROM
    dbo.FactInternetSales AS isales
JOIN
    dbo.DimProduct AS p ON isales.ProductKey = p.ProductKey
JOIN
    dbo.DimProductSubcategory AS psub ON p.ProductSubcategoryKey = psub.ProductSubcategoryKey
JOIN
    dbo.DimProductCategory AS pc ON psub.ProductCategoryKey = pc.ProductCategoryKey
GROUP BY
    pc.EnglishProductCategoryName
ORDER BY
    TotalSales DESC;

📊 Results:
  Execution: ✅ SUCCESS
  Rows Returned: 3
  Tokens Used: 943
  Latency: 5.32s
```

Evaluation

```
----------------------------------------------------------------------
Detailed Results by Query:
----------------------------------------------------------------------
 test_id  precision  recall  f1_score  relevant_retrieved
       1        1.0   1.000     1.000                   1
       2        1.0   1.000     1.000                   2
       3        1.0   1.000     1.000                   1
       4        1.0   1.000     1.000                   1
       5        1.0   1.000     1.000                   1
       6        1.0   1.000     1.000                   1
       7        1.0   1.000     1.000                   2
       8        1.0   1.000     1.000                   3
       9        1.0   1.000     1.000                   4
      10        1.0   0.500     0.667                   1
      11        1.0   0.500     0.667                   1
      12        1.0   0.667     0.800                   2
======================================================================
```

```
💰 TOKEN USAGE:
----------------------------------------------------------------------
  Baseline Avg: 4,134 tokens
  RAG Avg:      852 tokens
  Increase: -79.4%

⏱️  LATENCY:
----------------------------------------------------------------------
  Baseline Avg: 2.38s
  RAG Avg:      1.70s
```